

Documentation of English to Hindi Transformer Translation Model

This notebook contains the implementation of an English-to-Hindi neural machine translation system built using the Transformer architecture in PyTorch. Instead of using high-level libraries that hide most of the internal working, the notebook manually builds the important parts of the Transformer. Because of this, the project is not only about getting translations but also about understanding how modern NLP systems actually work internally.

Project Goal

The main objective of the project was to train a model capable of translating English sentences into Hindi sentences. The model was trained from scratch on a parallel English-Hindi dataset. The notebook focuses heavily on learning and experimentation rather than simply using pretrained APIs. A large part of the work involves building the architecture manually, preprocessing the dataset properly, handling tokenization, training the model, and evaluating translation quality.

Dataset and Preprocessing

The notebook uses a parallel dataset where each English sentence has a corresponding Hindi translation. Before training, the data goes through several cleaning steps. Invalid characters, URLs, duplicate lines, very noisy sentences, and badly aligned sentence pairs are removed. Sentence length filtering is also used to avoid extremely long or extremely short samples.

This preprocessing stage is important because translation models are highly sensitive to noisy data. Even small inconsistencies can reduce translation quality significantly, especially when the dataset size is not extremely large.

Tokenization and Vocabulary

Instead of using modern subword tokenizers like SentencePiece or BPE, the notebook uses custom regex-based tokenization. English words, punctuation, contractions, and Hindi Unicode characters are separated manually through pattern matching.

After tokenization, vocabularies are built separately for English and Hindi. Each word is mapped to an integer ID. Special tokens such as <pad>, <bos>, <eos>, and <unk> are also added. These tokens help the model understand where sentences begin, where they end, and how to handle unknown words.

Why the Transformer Architecture Was Used

Older sequence models like RNNs and LSTMs process words one by one. This makes training slower and also creates difficulty in remembering long-range dependencies. Transformers solve this problem using attention mechanisms.

The key idea behind the Transformer is that every word can directly interact with every other word in the sentence. This allows the model to capture context much more effectively. For translation tasks, this becomes extremely useful because the meaning of a word often depends on other distant words in the sentence.

Architecture of the Model

The notebook follows the classical encoder-decoder Transformer architecture introduced in the paper 'Attention Is All You Need'.

The encoder reads the English sentence and creates contextual representations of the input words. The decoder then generates Hindi words one token at a time while continuously attending to the encoder outputs.

The architecture includes embedding layers, positional encoding, multi-head attention, feed-forward neural networks, layer normalization, residual connections, and output projection layers. All these components are implemented manually instead of using a ready-made Transformer block.

Attention Mechanism

The most important concept used in the notebook is self-attention. During attention computation, every word compares itself with every other word and learns which words are important for understanding context.

For example, while translating the word 'bank', the model may look at nearby words like 'river' or 'money' to understand the correct meaning. This contextual understanding is one of the biggest reasons Transformers outperform older models.

Positional Encoding

Unlike RNNs, Transformers do not naturally understand sequence order because all words are processed in parallel. To solve this problem, positional encodings are added to word embeddings. These encodings help the model understand which word came first, second, third, and so on.

The notebook uses sinusoidal positional encoding similar to the original Transformer paper.

Training Process

The model is trained using teacher forcing. During training, the decoder is given the correct previous Hindi word instead of its own prediction. This stabilizes learning and helps the model converge faster.

Cross-entropy loss is used as the optimization objective along with label smoothing. The notebook also uses Adam optimization, learning-rate warmup, gradient clipping, mixed precision training, and gradient accumulation. These techniques help improve stability and training efficiency on limited GPU memory.

Decoding and Translation Generation

After training, the model generates Hindi translations using decoding strategies. The notebook implements both greedy decoding and beam search decoding.

Greedy decoding simply selects the most probable next word at each step. Beam search is more advanced because it keeps multiple possible sentence candidates during generation. This generally improves translation quality and sentence fluency.

Evaluation Metrics and Results

The notebook evaluates the model using BLEU score, chrF score, exact-match accuracy, perplexity, and validation loss.

The final reported results were approximately:

Training loss: 2.5261

Validation loss: 4.3955

Development BLEU score: around 12.9

Test BLEU score: around 13.8

Exact-match accuracy remained relatively low.

Although these scores are not state-of-the-art, they are still meaningful because the model was trained from scratch on a relatively limited dataset. Translation tasks are extremely difficult, especially for morphologically rich languages like Hindi.

Observations from the Model

The notebook clearly shows that the Transformer successfully learned important translation patterns and sentence structures. The generated outputs indicate that the model understands partial semantic relationships and word ordering.

However, there are still visible limitations. Some outputs may contain grammatical mistakes, repeated words, or incorrect translations. This is expected when training a Transformer from scratch on smaller datasets without pretrained language knowledge.

Possible Improvements

Several improvements could significantly increase translation quality. The biggest improvement would be using subword tokenization such as SentencePiece or BPE. Fine-tuning pretrained multilingual models like mBART, mT5, or IndicTrans would also improve performance massively.

Increasing dataset size, using larger model dimensions, training for more epochs, and improving regularization could further improve results.

Overall Conclusion

Overall, the notebook is a strong educational implementation of a Transformer-based machine translation system. The project demonstrates understanding of deep learning, attention mechanisms, sequence modeling, optimization, and neural machine translation workflows.

The most impressive aspect of the notebook is that the architecture is implemented manually rather than relying completely on abstraction libraries. This makes the project valuable from a learning perspective because it forces understanding of how Transformer models actually work internally.